

Clase 137 — Descubrimiento de vulnerabilidades en código

Parte: 5 — **Explotación de sistemas y binarios** · Fuente: *Dowd, McDonald, Schuh, The Art of Software Security Assessment* 🕒 Duración estimada: **120 min** · Nivel: **Avanzado**

🎯 Objetivo

Aprender a **encontrar vulnerabilidades** de forma sistemática combinando auditoría manual de código, análisis estático automatizado (SAST) y razonamiento sobre superficies de ataque. Complementa el fuzzing (clase 136) con la revisión dirigida que detecta bugs lógicos y patrones peligrosos que el fuzzer no alcanza fácilmente. Cerrarás con la práctica de **divulgación responsable**.

⚠️ **Ética:** audita código propio, open source o con autorización. Reporta de forma responsable (coordinated disclosure), nunca publiques 0-days de terceros sin proceso.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Modelar** la superficie de ataque de un componente (entradas, confianza, límites).
- 2. **Auditar** código C/C++ buscando patrones peligrosos (memoria, enteros, formato).
- 3. **Aplicar** SAST (clang-analyzer, cppcheck, Semgrep, CodeQL) e interpretar hallazgos.
- 4. **Priorizar** por explotabilidad e impacto.
- 5. **Redactar** un reporte y seguir un proceso de divulgación responsable.

📖 Temas

#	Tema	Por qué importa
1	Superficie de ataque y fuentes de entrada	Dónde entran los datos no confiables
2	Patrones peligrosos en C/C++	Dónde suelen estar los bugs
3	Taint / seguimiento de datos	Del input a la operación peligrosa
4	SAST (cppcheck, clang, Semgrep)	Automatizar la detección
5	CodeQL	Consultas semánticas de vulns
6	Falsos positivos/negativos	Interpretar con criterio
7	Priorización por explotabilidad	Enfocar el esfuerzo
8	Reporte y disclosure	Cerrar el ciclo con ética

Definiciones y características

- **Superficie de ataque:** conjunto de puntos donde entran datos no confiables. *Clave:* prioriza parsers, IPC, red y ficheros.
- **Análisis de taint:** rastrea datos controlados por el usuario hasta operaciones sensibles (`memcpy` , índices, `system`). *Clave:* fuente→sumidero.
- **SAST:** análisis estático de seguridad sobre el código. *Clave:* rápido y escalable, pero genera falsos positivos.
- **CodeQL:** motor que trata el código como base de datos consultable. *Clave:* permite escribir queries para clases de bugs (p. ej. desbordamientos).
- **Divulgación responsable:** reportar al vendor y coordinar el arreglo antes de publicar. *Clave:* respeta plazos y a los usuarios.
- **Explotabilidad:** qué tan factible es convertir el bug en impacto real. *Clave:* guía la priorización (no todo bug es crítico).

Herramientas y preparación

```
sudo apt install -y cppcheck clang-tools
pip install semgrep
# CodeQL CLI: descargar de github.com/github/codeql-cli-binaries
```

Laboratorio guiado

Entorno propio.

1. Modela la superficie de ataque de un proyecto C pequeño: lista funciones que reciben datos externos (argv, ficheros, red) y márcalas como fuentes.
2. Ejecuta SAST y compara resultados:

```
bash cppcheck --enable=all --inconclusive src/ 2> cppcheck.txt scan-build make # clang static analyzer semgrep --config p/c src/
```

3. Sigue una advertencia real (p. ej. `memcpy` con tamaño controlado) desde la fuente del dato hasta el sumidero, confirmando si es explotable o falso positivo.
4. Escribe una consulta CodeQL sencilla que localice llamadas a `strcpy` con origen no acotado (o parte de una query de ejemplo del repo de CodeQL) y ejecútala sobre la base del proyecto.
5. Prioriza los hallazgos en una tabla: bug, fuente, sumidero, explotabilidad (alta/media/baja), impacto.
6. Redacta un mini-reporte de una vulnerabilidad como si fueras a enviarlo al mantenedor: descripción, PoC mínima, versiones afectadas, mitigación y una propuesta de plazo de divulgación.

Ejercicios

1. Dibuja el diagrama de superficie de ataque de un servicio que lee de red.
2. Encuentra manualmente un `sprintf` sin límite y explica el riesgo.

3. Compara los hallazgos de cppcheck vs Semgrep en el mismo código.
4. Escribe una regla Semgrep que detecte `gets()`.
5. Clasifica 5 hallazgos por explotabilidad e impacto.
6. Redacta el cuerpo de un reporte de divulgación responsable.

Reto verificable

Audita un proyecto C pequeño (propio o open source con permiso), identifica una vulnerabilidad real, demuéstrela con una PoC mínima y redacta el reporte de divulgación.

Criterio de aceptación: entregas la ubicación exacta del bug (archivo:línea), una PoC que lo dispara y un reporte con impacto, versiones y mitigación propuesta.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
SAST ahoga en falsos positivos	Filtra por severidad y valida con taint manual
No encuentras nada	No modelaste bien la superficie de ataque; empieza por los parsers
CodeQL no compila la base	La query necesita una DB creada con <code>codeql database create</code>
Reportas sin PoC	Debilita el reporte; incluye reproducción mínima
Publicas un 0-day de terceros	Viola la ética; sigue disclosure coordinada

Preguntas frecuentes

 **¿SAST reemplaza al fuzzing?** No: SAST ve patrones sin ejecutar; fuzzing ejecuta y halla bugs de runtime. Son complementarios.

 **¿Cómo reporto responsablemente?** Contacta al vendor/security.txt, aporta PoC, acuerda plazo (p. ej. 90 días) y coordina la publicación.

 **¿Todo hallazgo es una CVE?** No: muchos son de baja explotabilidad o requieren condiciones irreales; prioriza.

Referencias

- Dowd, McDonald, Schuh. *The Art of Software Security Assessment*. Addison-Wesley.
- CodeQL — <https://codeql.github.com/>
- Semgrep — <https://semgrep.dev/>
- security.txt / disclosure — <https://securitytxt.org/>

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Siguiente clase

Clase 138 - Desarrollo de exploits moderno